

Chiffrage OEM — Rings & Segmentation

What the OEM pricing calculator does, how it is built, and what just changed in it — so the whole team can find its way around, today and six months from now (`chiffrage_oem.py`).

STATUS

In production — `chiffrage_oem.py` has been the official build since July 8, 2026

DOCUMENTED VERSION

`chiffrage_oem.py` — 3,258 lines

DATE

July 8, 2026

AUTHORS

Dylan Carlier — initial design: N. Richet, R. Demeure

§1 Purpose of this document

Chiffrage OEM computes unit prices for sealing rings and segments (PTFE, PEEK) sold to compressor manufacturers. This document explains how it works, what it lets you do, and what just changed: the pricing logic, the data it handles, the interface we reworked, and the bugs we fixed along the way.

The goal: a future user, a future administrator, or ourselves in six months, should be able to find their way around without having to re-read the whole codebase.

§2 General overview

In practice: we start from a 2021 catalog price — the **PL21** — specific to a given part (a reference, a diameter, a geometry), and roll it forward in time. It is entered by hand at every calculation: the application does not store a parts catalog, only the pricing rules negotiated per client.

Each client has its own trajectory: a series of cumulative annual increases (2022, 2023, 2025, 2026...) reflecting cost changes, followed by a negotiated discount that brings the price down to what is actually invoiced. Fifteen clients already run in the tool by default — *Client A*, *Client B*, *Client D*, *Client G*, *Client H* — spread across two materials. Clients can be added, removed, or have their rates changed without touching a single line of code.

§3 Pricing logic

All the logic fits in a single formula:

$$\text{price_final} = \lceil \text{PL21} \times \prod (1 + \text{rate}_{\text{year}}) \times (1 + \text{discount}) \rceil$$

- **PL21** — 2021 catalog price, entered as a plain number (620), an operation (600 + 50), or a percentage change (620 + 3%).
- **rate_{year}** — increase specific to the (material, client) pair, applied successively in chronological order.

- **discount** — rate negotiated with the client, stored as a negative value (-0.70 for -70%).
- [] — the result is rounded up to the nearest whole euro (final prices are always displayed with no decimals, e.g. 207 €); intermediate prices keep two decimals.

An example speaks better than a formula — **Client A, PTFE, PL21 = 620 €**

Step	Rate	Price after
PL21	—	620.00 €
2022	+0.0%	620.00 €
2023	+7.0%	663.40 €
2025	+0.0%	663.40 €
2026	+4.0%	689.94 €
Discount	-70.0%	207 €

§4 Data model

Nothing complicated on the storage side: everything lives in a single file, `chiffrage_data.json` — no database to install or maintain.

```
{
  "augmentations": {
    "PTFE": { "Client A": { "2022": 0.00, "2023": 0.07, ... }, ... },
    "PEEK": { ... }
  },
  "remises": {
    "PTFE": { "Client A": -0.70, ... }, "PEEK": { ... }
  },
  "remarques": { "Client C": "Client B for Client C: Client C price - 20%", ... },
  "historique": [ { ts, pl21, matiere, client, prix_final }, ... ], // last 500 entries
  "audit_log": [ { ts, action, matiere, client, details }, ... ], // last 1,000 entries
  "admin_hash": "pbkdf2_sha256$..."
}
```

File location

- **Development machine** — next to the source script.
- **User machine** (once the app is installed) — `%APPDATA%\CPI Howden\Chiffrage OEM\`. It had to be moved there: Program Files refuses write access to standard accounts, and before this fix the app crashed on the very first click on "Calculate". An automatic migration recovers data if an older install had left it elsewhere.

§5 Application architecture

Nothing exotic on the technical side:

- **Language / framework** — Python 3 + PyQt6, Fusion style (consistent rendering across Windows/Mac/Linux).
- **Excel export** — optional, via openpyxl; disables cleanly if the module is absent.
- **PDF export** — QTextDocument + QPrinter, no external dependency.
- **Persistence** — a single JSON file, no locking: suited to single-workstation use, not concurrent access (see §10).

Code organization

Everything lives in a single file, `chiffrage_oem.py` (3,258 lines as of today), split into 12 classes: `MainWindow` (main window), `ClientDetailPanel` (detail panel and waterfall), `WaterfallChart` (hand-painted visualization, no charting library), `ClientListItem`, `StatCard`, `Toast`, `TitlebarThemer`, and the five admin dialogs (`EditClientDialog`, `AddClientDialog`, `ChangePasswordDialog`, `AuditLogDialog`, `HistoriqueDialog`).

§6 Features

6.1 — Input & calculation

- Material selector (`PTFE` / `PEEK` by default, extensible without coding).
- PL21 field accepting three formats, with a live preview of the result: plain number (`620`), operation (`600 + 50`), percentage change (`620 + 3%`).
- "Calculate" (or Enter) prices every visible client — respecting the active search filter — and logs each calculation to the history.
- Client search filter (`Ctrl+F`), keyboard navigation `↑` / `↓` in the list.

6.2 — Calculation visualization

The detail panel tells the story of the calculation rather than just giving a number: the PL21 at the bottom, each increase stacking on top of it, then the discount eating into a share of the total — visible at a glance thanks to a hatched gradient. Three small cards summarize the essentials (total increase, discount, final price), and the final price copies with one click when it needs to be shared quickly.

6.3 — Administration (password-protected)

- Add / remove a client — removal takes the client out of every material at once.
- Add a material — inherits the existing client list, rates initialized at 0%.
- Edit a client's annual rates and remark — years can be added and removed dynamically.
- Edit a client's discount — the only action reachable without a password from the detail panel, a frequent day-to-day gesture.
- Change the administrator password.

6.4 — Exports

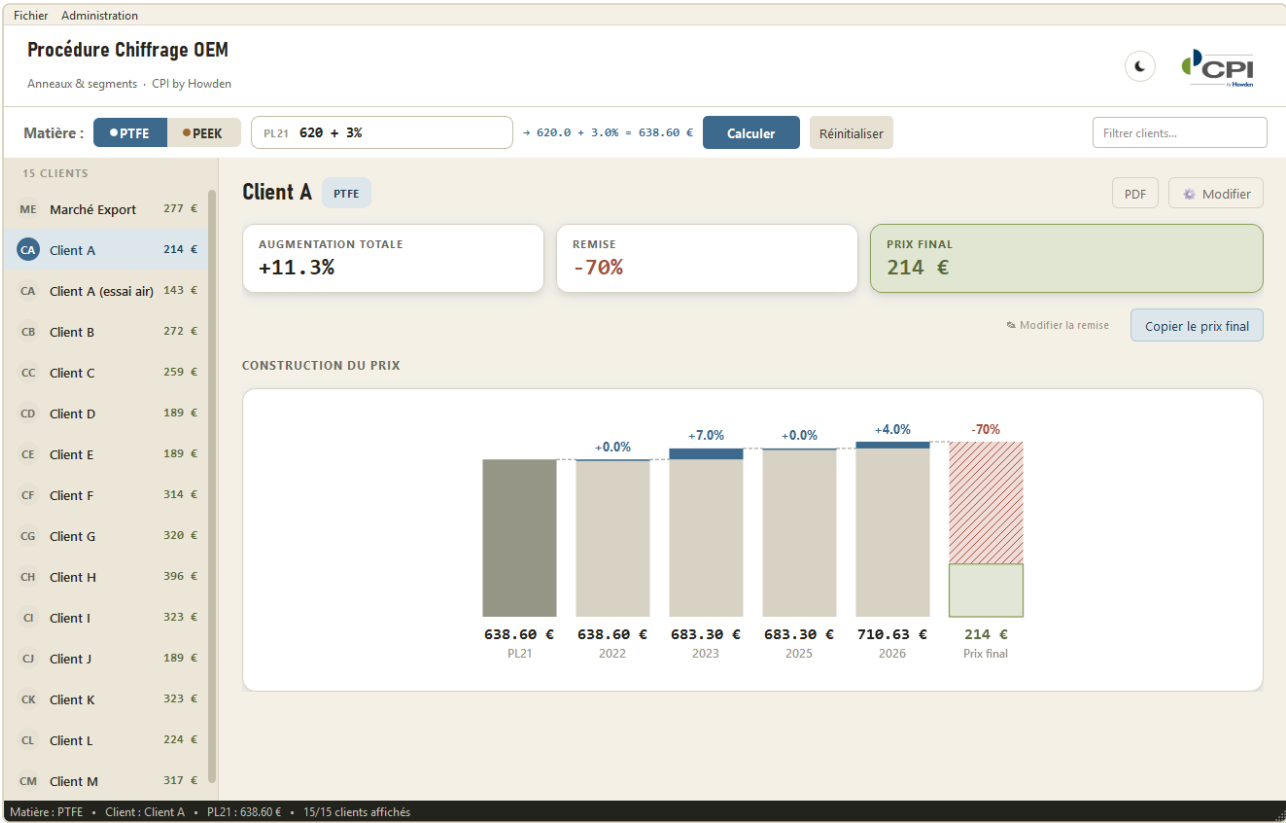
- **CSV** — ";" separator, UTF-8 with BOM encoding (Excel compatibility).
- **Excel (.xlsx)** — formatted with material colors, final prices in green.
- **PDF** — detailed sheet per client, or a summary for an entire material.

6.5 — Traceability

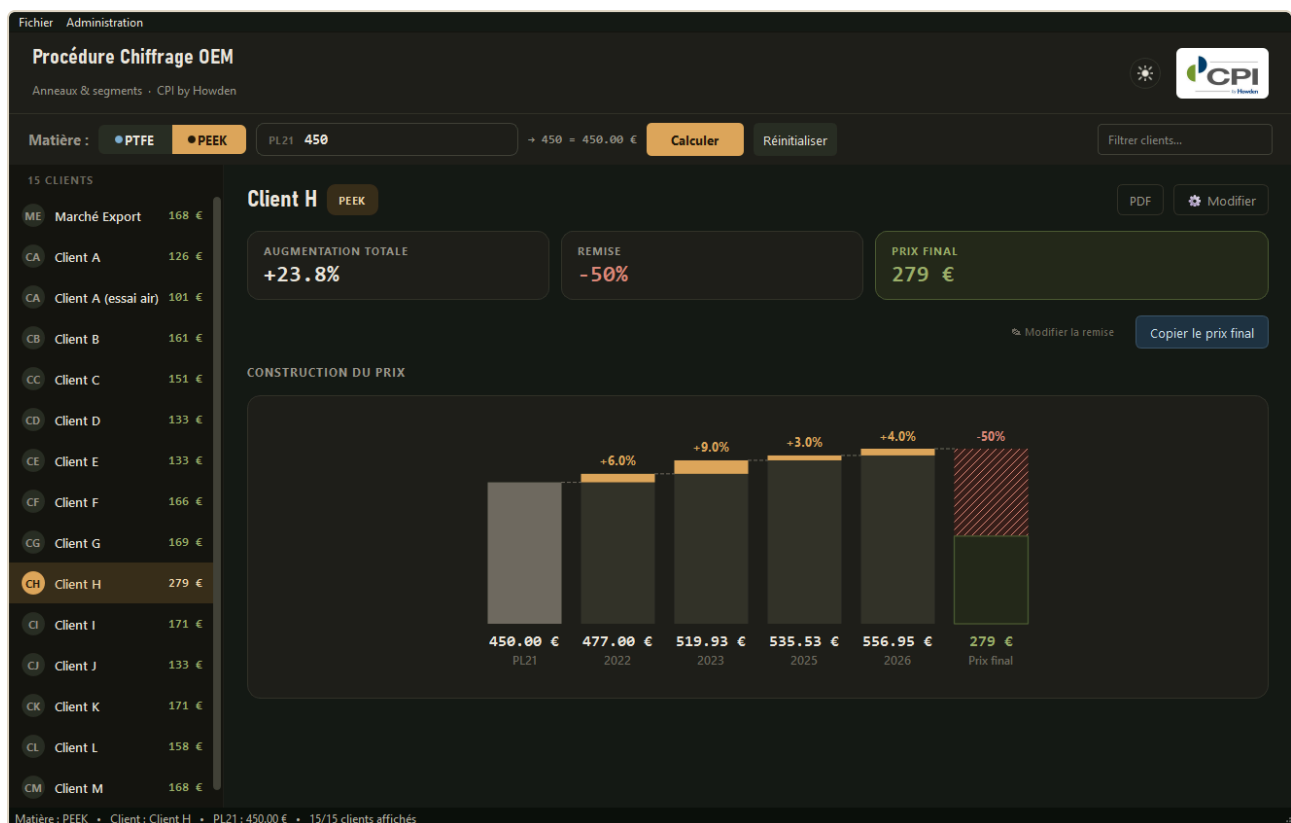
- Calculation history (date, material, client, PL21, resulting price) — last 500 entries, viewable and clearable.
- Administrative action audit log — last 1,000 entries, read-only.

§7 Screenshots

Rather than describe the interface, better to show it: the main screen in light and dark mode, the administration dialog, and a PDF export generated by the app itself — not a mock-up.



Light theme — PTFE material. PL21 entered as a percentage (620 + 3%); calculation waterfall, summary cards, and a discount of -70%.



Dark theme — PEEK material. All 15 clients priced at once in the sidebar; amber accent specific to PEEK on the discount and final price.

Client : Client A | Matière : PTFE

Taux annuels d'augmentation

Année 2022 : ✕

Année 2023 : ✕

Année 2025 : ✕

Année 2026 : ✕

+ Ajouter une année

Remarque (optionnel)

Enregistrer

Cancel

Administration — edit a client. Annual rates addable/removable dynamically, free-text remark; reached via "⚙ Edit", password-protected.

Chiffrage OEM — Client A

Matière : PTFE | Date : 09/07/2026 | PL21 : 638.60 €

Année	Taux appliqué	Prix après (€)
2022	+0.0%	638.60 €
2023	+7.0%	683.30 €
2025	+0.0%	683.30 €
2026	+4.0%	710.63 €
Remise	-70%	214 €

Prix unitaire final : 214 €

Client PDF export. Sheet generated by "PDF" in the detail panel — table of calculation steps and final price highlighted.

§8 Changelog — interface polish

This iteration was spent polishing the interface: animations and transitions designed to stay within the "technical sheet" spirit already in place (warm paper tones, PTFE/PEEK accents) — not added just for the sake of it.

- **Animated waterfall** — the columns "grow" from the base one after another, slightly staggered, instead of appearing all at once: the formula builds itself in front of the user's eyes.
- **Animated counters** — the final price and total increase roll up to their new value (odometer effect, ~400 ms).
- **Client-change fade** — the detail panel fades in (160 ms) instead of a hard swap.
- **Toast notification** — a sliding, self-dismissing confirmation for copying the final price.
- **Theme cross-fade** — light/dark switch via screenshot capture + fade (260 ms), no more flicker.
- **Material color transition** — the Calculate button slides from PTFE steel blue to PEEK amber (240 ms) on material change.
- **Waterfall hover** — highlighted column and tooltip detailing the step (price before → rate → price after).
- **Themed Windows title bar** — dark in dark mode; thin scrollbars in the theme's colors.
- **Enlarged waterfall** — height 210→270 px, columns 60→76 px, price font 8→12 pt, for better readability of the calculation detail. Enlarged after review: it felt too dense to read the detail quickly.
- **"Flash" halo on each client row on recalculation** — implemented, then removed. Removed after review: it looked more like a display glitch than a real signal.

§9 Reliability & security — findings from the code review

We re-read all of `chiffnage_oem.py` looking for anything that could go wrong: unhandled errors, unchecked inputs. Eight issues stopped us — all fixed, and verified by 18 automated checks plus a full replay of the interface scenario, without breaking anything along the way.

#	What we found	What could happen	What we did
01	The "base – discount%" formula let a zero or negative PL21 through	Division by zero at calculation time, or worse: a negative price displayed as if nothing were wrong	The sign of the result is now checked before being accepted
02	Writing the data file was not atomic	A crash or power outage mid-write truncates the JSON — and with it, every negotiated discount and remark	We now write to a temporary file, then swap it in one operation (<code>os.replace</code>)
03	A corrupted file was replaced without warning anyone	Invisible data loss — no trace, no warning	The file is renamed to <code>.corrompu.bak</code> and kept, with a clear message on startup
04	Client names, materials, or remarks were not escaped in exported PDFs	A single "&" or "<" in a name is enough to break the document's rendering	All free text now goes through systematic HTML escaping
05	CSV / Excel exports were vulnerable to formula injection	A client name starting with <code>=</code> , <code>+</code> , <code>-</code> or <code>@</code> executes as a formula when opened in Excel (CWE-1236)	A neutralizing prefix is now added automatically on the affected fields
06	Exporting to a file already open elsewhere (Excel, a PDF reader...)	Surfaced a generic, unreadable error — "an unexpected error occurred..."	The message now clearly states what most likely happened
07	Adding a client right after removing the last one of a material	Crashed the app (<code>IndexError</code>), barely caught by the global filter	The app now falls back cleanly to another existing material
08	Adding a material on a dataset emptied by hand	Same potential issue (<code>IndexError</code>), if someone edits the JSON directly	A defensive guard now covers this case

§10 Known limitations & recommendations

Administrator password — at the time this document was written, the SHA-256 hash was unsalted and the default password was written in plain text in the code. This has since been hardened: the hash is now salted (PBKDF2-HMAC-SHA256) and a random password is generated at first launch instead of a fixed default. For a single-workstation app, the real security boundary remains physical access to the machine rather than this password.

- **No concurrent access handling** — a single JSON file, no locking. Two people modifying the data at the same time would overwrite each other. No consequence as long as the tool stays on a single workstation, which is the case today.

- **No versioned test suite** — this iteration's checks (interface scenario, error checks) were written for the occasion and run by hand, without being kept in the repository.

§11 Roadmap

The following follows directly from the limitations in §10 — nothing is added here that does not respond to an actual finding from this review.

1 — Immediate, before the next release

- Change the default administrator password on the production machine.
- Manually replay the full scenario (calculation, export, theme, administration) on the target machine before cutover.

2 — Short term

- Add the verification scenarios written during this iteration (PL21 validation, atomic writes, exports, edge cases) to the repository, instead of leaving them as one-off scripts.
- Add a minimal check before commit/merge (replay of the interface scenario).

3 — Conditional, if usage grows

- Harden authentication (salted hash, per-user account) — done, see §10 — the day the tool moves beyond its current single-workstation use.
- Plan for storage that tolerates concurrent access (at minimum, file locking), should several workstations ever need to touch the same data.

§12 Dependencies & distribution

- **Required** — Python 3, PyQt6.
- **Optional** — openpyxl (disables Excel export if absent, without error).
- **Packaging** — PyInstaller (`--onefile --windowed`), Inno Setup installer, dynamic versioning via `release.ps1`.

§13 Glossary

Term	Definition
PL21	Reference catalog price (2021 list), entered per part at each calculation.
Material	Material family (PTFE, PEEK...); carries its own rates and discounts per client.
Annual increase rate	Cumulative annual increase applied to the PL21, specific to each (material, client) pair.
Discount	Final negotiated rate, brings the price down to the agreed commercial level.
Waterfall	Step-by-step visualization of how the price is built.